Python Strings

Part 1: Understanding Strings

Video #4.1 | PyMaster India

Creation · Indexing · Slicing · Immutability · Basic Operations

■ String Creation	**■ Indexing**	**✂■ Slicing**
Single, double & triple quotes	Positive & negative indexing	start, stop, step syntax
■ Immutability	**■■ Basic Operations**	**■ Escape Characters**
Why strings can't be changed	Concat, repeat, membership, length	\n, \t, \\ and friends

Rajeev Shivah Full-Stack Developer & Data Scientist | codeWithShivah | learn.rajeevshivah.me

© Rajeev Shivah | codeWithShivah | Not for redistribution

Table of Contents

1.	What is a String?	3
2.	Creating Strings	3
3.	String Indexing — Positive & Negative	4
4.	String Slicing	5
5.	Immutability of Strings	7
6.	Basic String Operations	8
7.	Escape Characters	9
8.	Practice Exercises	10
9.	Quick Reference Cheat Sheet	11

© Rajeev Shivah | codeWithShivah | Not for redistribution

1. What is a String?

The fundamental data type for text in Python

A **string** is a **sequence of characters** enclosed in quotes. It is one of Python's built-in data types (type: **str**) used for all text — names, sentences, file paths, messages, and more.

```
# Examples of strings
name = "Rajeev"
city = 'Varanasi'
message = "Python is fun!"
empty = "" # empty string — still a string
print(type(name)) # <class 'str'>
print(len(name)) # 6
```

■ Key Insight

Python stores a string as an ordered sequence of Unicode characters.

Every character has an index position starting from 0.

Strings are **immutable** — once created, they cannot be changed in place.

2. Creating Strings

Single, double, triple quotes & when to use each

Style	Syntax	Use Case
Single quotes	`'Hello'`	`Simple strings, no apostrophes inside`
Double quotes	`"Hello"`	`When string contains an apostrophe`
Triple single	`'''Multi\nline'''`	`Multi-line strings, docstrings`
Triple double	`"""Multi\nline"""`	`Multi-line strings, docstrings`

2.1 Quote Style Examples

```
s1 = 'Hello, World!'
# Double quotes — handy when string contains apostrophe
s2 = "It's Python!"
# Triple quotes — spans multiple lines, preserves newlines
s3 = """Line 1
Line 2
Line 3"""
s4 = '''
Dear Student,
Welcome to PyMaster India.
'''
```

■■ Common Mistake

Mixing quote types without escaping causes a SyntaxError:

■ s = 'It's wrong' → SyntaxError

■ s = "It's correct" → OK

■ s = 'It\'s correct' → OK (escaped)

3. String Indexing

Positive indexing & negative indexing

Every character in a string has an **index** (position number). Python supports **positive** (left→right) and **negative** (right←left) indexing.

P	Y	T	H	O	N
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

Blue row = positive index (0 → 5) Yellow row = negative index (–1 → –6)

3.1 Accessing Characters

```
s = "PYTHON"
# Positive indexing
print(s[0]) # P — first character
print(s[1]) # Y
print(s[5]) # N — last character
# Negative indexing (count from the end)
print(s[-1]) # N — last character
print(s[-2]) # O
print(s[-6]) # P — same as s[0]
```

3.2 Index Out of Range

```
s = "PYTHON" # length = 6
print(s[6]) # ■ IndexError: string index out of range
print(s[-7]) # ■ IndexError
# Valid positive range: 0 to len(s)-1
# Valid negative range: -1 to -len(s)
```

■ Quick Rule

Valid positive indices: 0 to len(s) – 1

Valid negative indices: –1 to –len(s)

Accessing len(s) directly always raises IndexError.

4. String Slicing

Extract sub-strings with start, stop, step

Slicing lets you extract a **portion** of a string. Syntax: **s[start : stop : step]**. Characters from index *start* up to (but **not including**) *stop* are returned, jumping by *step*.

```
s[ start : stop : step ]
```

Parameter	Default	Meaning
start	0	Index to start from (inclusive)
stop	len(s)	Index to stop at (exclusive)
step	1	How many characters to jump each time

4.1 Basic Slicing

```
s = "PYTHON"
# P Y T H O N
# 0 1 2 3 4 5
print(s[0:3]) # PYT
print(s[2:5]) # THO
# Omitting start or stop
print(s[:3]) # PYT — same as s[0:3]
print(s[3:]) # HON — same as s[3:6]
print(s[:]) # PYTHON — full copy
```

4.2 Slicing with Step

```
s = "PYTHON"
print(s[::2]) # PTN — every 2nd character
print(s[1::2]) # YHN — start at 1, every 2nd
# Negative step reverses the string
print(s[::-1]) # NOHTYP
```

4.3 Negative Indices in Slicing

```
s = "PYTHON"
print(s[-3:]) # HON — last 3 characters
print(s[:-3]) # PYT — all except last 3
print(s[-4:-1]) # THO
```

4.4 Slice Quick Reference

Expression	Result	Explanation
s[:]	PYTHON	Full copy
s[::-1]	NOHTYP	Reverse string
s[::2]	PTN	Every 2nd char
s[:3]	PYT	First 3 chars
s[-3:]	HON	Last 3 chars
s[1:-1]	YTHO	Strip first & last

■ Slicing Never Raises IndexError

Unlike direct indexing, out-of-range slice values are silently clamped.

s = "PY" → s[0:100] returns "PY" (no error)

5. Immutability of Strings

Why you can't change a string character in place

Strings in Python are **immutable**: after creation, individual characters **cannot be changed**. Any operation that 'modifies' a string actually creates a **brand-new string** in memory.

5.1 Attempting to Mutate — TypeError

```
s = "PYTHON"
s[0] = 'J' # ■ TypeError: 'str' object does not support item assignment
```

5.2 Correct Approach — Build a New String

```
s = "PYTHON"
new_s = 'J' + s[1:]
print(new_s) # JYTHON
s2 = "I love Java"
new_s2 = s2.replace("Java", "Python")
print(new_s2) # I love Python
```

■■ Performance Tip

s += "x" inside a loop creates a new string on every iteration — expensive.

Better: use a list to collect, then join at the end.

parts = []; parts.append('x'); result = ''.join(parts)

6. Basic String Operations

Concatenation, repetition, membership, length

```
# Concatenation ( + )
first = "Rajeev"
last = "Shivah"
full = first + " " + last
print(full) # Rajeev Shivah
# Repetition ( * )
line = "-" * 30
print(line) # ------------------------------ (30 dashes)
print("Ha" * 3) # HaHaHa
# Length ( len() )
print(len("Python")) # 6
print(len("")) # 0
# Membership ( in / not in )
s = "PyMaster India"
print("Master" in s) # True
print("Java" not in s) # True
# Iteration
for ch in "PY":
print(ch, end=" ") # P Y
```

Operator	Description	Example	Output
+	Concatenate	"Hi" + " World"	"Hi World"
*	Repeat	"AB" * 3	"ABABAB"
in	Substring test	"Py" in "Python"	True
not in	Not present	"Go" not in "Py"	True
len()	Length	len("Python")	6

7. Escape Characters

Special characters inside strings

Sequence	Meaning	Example	Output
\n	Newline	"Hi\nWorld"	Hi (newline) World
\t	Tab	"Hi\tWorld"	Hi World
\\	Backslash	"C:\\Users"	C:\Users
\"	Double quote	"She said \"Hi\""	She said "Hi"
\'	Single quote	'It\'s fine'	It's fine
\r	Carriage return	(terminal output)	—

```
print("Hello\nWorld") # Hello
# World
print("Name:\tRajeev") # Name: Rajeev
print("C:\\Users\\Rajeev") # C:\Users\Rajeev
```

8. Practice Exercises

15 problems covering all Part 1 topics

■ Beginner

Q1. Create a string with your name. Print its length and data type.

Q2. Print the first and last character of the string 'DataScience'.

Q3. Using negative indexing, print the 3rd character from the end of 'PyMaster'.

Q4. Slice 'PROGRAMMING' to extract only 'GRAM'.

Q5. Reverse the string 'India' using slicing.

■ Intermediate

Q6. Check whether 'Python' is present in 'I love Python programming'.

Q7. Concatenate 'Hello' and 'World' with a space between them.

Q8. Repeat '*' 20 times to print a separator line.

Q9. Extract every alternate character from 'ABCDEFGHIJ'.

Q10. From 'PyMaster India', extract 'India' using slicing.

■ Think & Apply

Q11. Try s[0] = 'X' on any string. Observe and explain the error you get.

Q12. Write a print statement that outputs: He said "Hello" (include the quotes).

Q13. Print a Windows file path: C:\Users\Rajeev\Desktop using escape characters.

Q14. What does s[1:-1] do? Test it on at least three different strings.

Q15. Can you find a slice expression that returns an empty string from 'PYTHON'? Find two different ways.

■ How to Practice

Attempt every problem in a .py file before checking any solution.

For Think & Apply questions — write your explanation as a comment in the code.

9. Quick Reference Cheat Sheet

Video #4.1 — Everything on one page

String Creation

```
s = 'hello'
s = "hello"
s = """multi-line"""
```

Basic Operations

```
"Hi" + " World" # concat
"AB" * 3 # repeat
"x" in s # membership
len(s) # length
```

Indexing

```
s[0] # first char
s[-1] # last char
s[n] # nth char
```

Escape Characters

```
"\n" # newline
"\t" # tab
"\\" # backslash
"\"" # double quote
```

Slicing

```
s[a:b] # index a to b-1
s[:3] # first 3
s[-3:] # last 3
s[::2] # every 2nd
s[::-1] # reverse
```

Immutability

```
# ■ s[0] = 'X' — TypeError
# ■ s = 'X' + s[1:] — new string
```

codeWithShivah learn.rajeevshivah.me | PyMaster India — Video #4.1